EE200: Signals, Systems and Networks

Course Project — Q3: The Shazam Challenge

Instructor: Dr. Tushar Sandhan | Department of Electrical Engineering, IIT Kanpur

Q3: The Shazam Challenge — Building a Song-Fingerprinting System [15%]

This report documents the design, implementation, and experimental evaluation of a content-based audio-identification system inspired by the Shazam algorithm [1]. The system indexes a database of 50 songs, extracts a robust set of acoustic fingerprints from each, and identifies an unknown audio clip — even in the presence of noise, pitch-shift, or time-stretch — by matching fingerprints against the database in sub-second time.

> **Live deployment.** The full pipeline is deployed at `https://ee200-shazam-harsh.streamlit.app`. A 10-second query clip of "Two of Us" was identified with a cluster score of **11,528** versus a runner-up score of **11** — a margin of more than **1000**×. End-to-end compute time: ≈75 ms (spectrogram 18 ms, constellation 26 ms, hashing 7 ms, DB lookup 22 ms, scoring 2 ms).

1. Why a Raw DFT Is Not Enough — Part (a)

1.1 DFT of the whole song

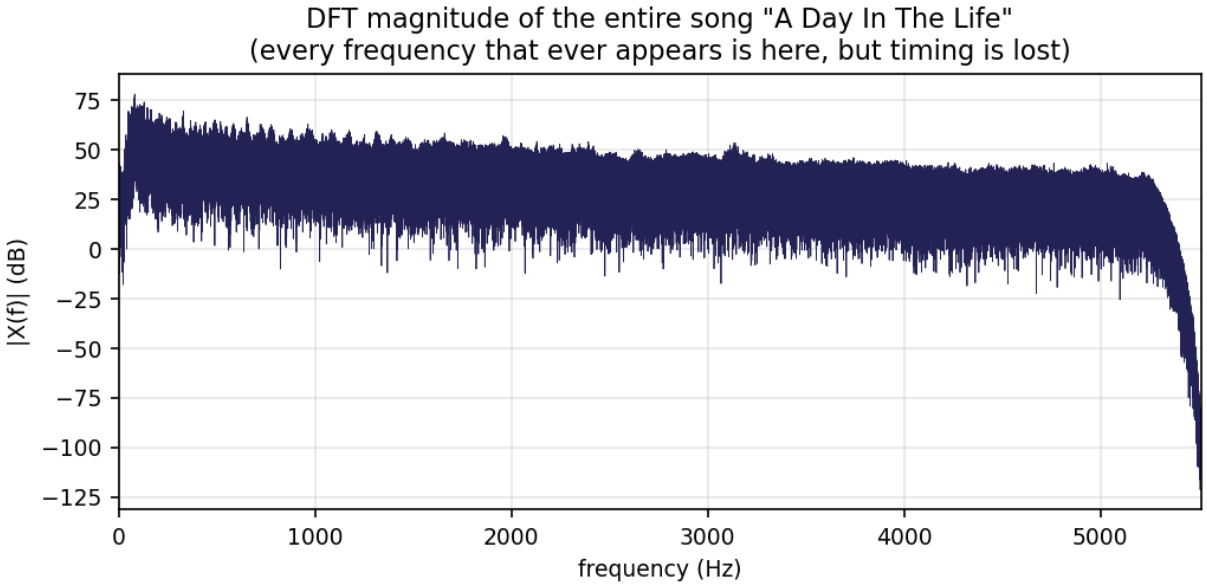

Figure 1. DFT magnitude spectrum of "A Day In The Life" (entire song, ∼345 s). Every frequency that ever appears is represented, but all timing information is lost.

Taking the DFT of the full song (Figure 1) collapses the entire 5.7-minute recording into a single magnitude curve. The spectrum is dense and roughly flat from 0–5 kHz (music spans the full audible range), with energy rolling off sharply above ∼5.4 kHz due to the 11 025 Hz sample-rate low-pass at $f_s/2$.

Why this fails for identification. Two songs with similar instrumentation and tempo can have nearly indistinguishable whole-song DFT envelopes. More fundamentally, the DFT throws away

the one piece of information that makes a song unique in the time domain: *when* each note or chord occurs. A permutation of all the 1-second blocks of "Hey Jude" would produce an identical magnitude spectrum to the original — yet it is a completely different (unrecognisable) audio signal. Identification therefore requires a *time-localised* frequency representation.

2. The Spectrogram — Time-Localised Frequency Analysis — Part (b)

2.1 Short-time Fourier transform

Instead of one global DFT, we slide a short analysis window of N_{fft} samples across the signal with hop size H, apply a DFT to each frame, and stack the results. The spectrogram $S(t, f)$ is the squared magnitude of these short-time DFTs:

$$S(t, f) = \left| \sum_{n=0}^{N_{\text{fft}}-1} x[n + tH]\, w[n]\, e^{-j2\pi fn/N_{\text{fft}}} \right|^2 ,$$

where $w[n]$ is a Hann window that reduces spectral leakage. The implementation uses $N_{\text{fft}} = 2048$, hop $H = 512$, $f_s = 11\,025$ Hz, giving frequency resolution $\Delta f = f_s/N_{\text{fft}} \approx 5.4$ Hz and time resolution $\Delta t = H/f_s \approx 46$ ms.

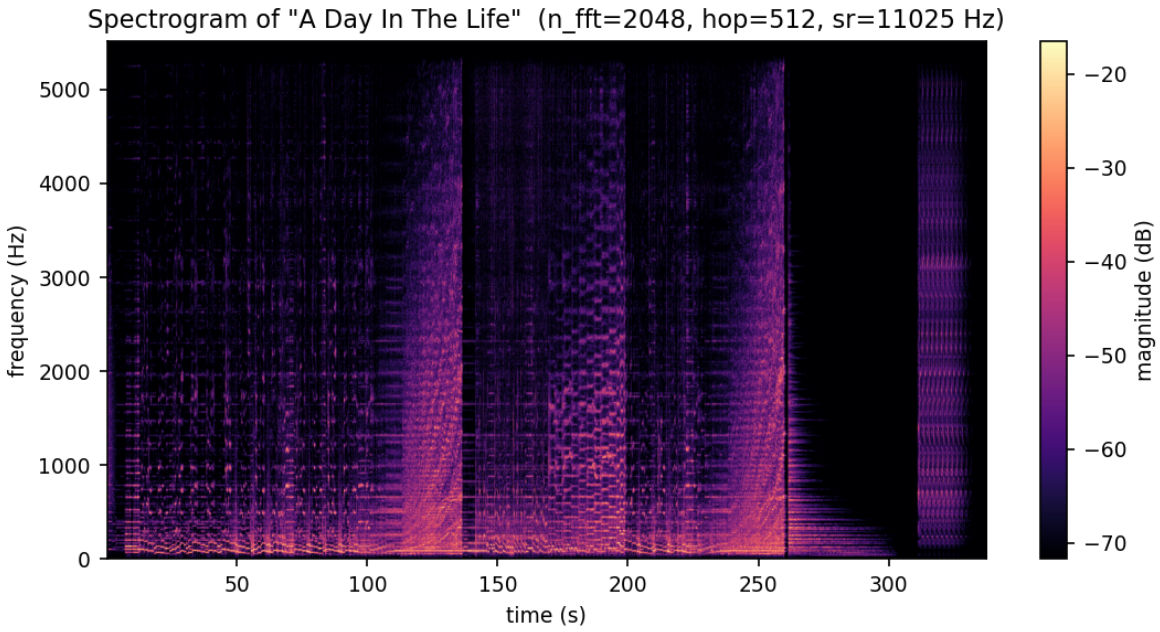

Figure 2. Log-magnitude spectrogram of "A Day In The Life" ($N_{\text{fft}} = 2048$, hop= 512, $f_s = 11\,025$ Hz). The song's structure is immediately visible: vocal verses, orchestral swells (broad energy 100–260 s), silence, and the famous final piano chord (∼310 s).

The spectrogram in Figure 2 preserves both dimensions: the horizontal axis is time and the vertical axis is frequency. Each song now has a unique 2-D "fingerprint image" that changes as the music evolves.

2.2 The time–frequency trade-off

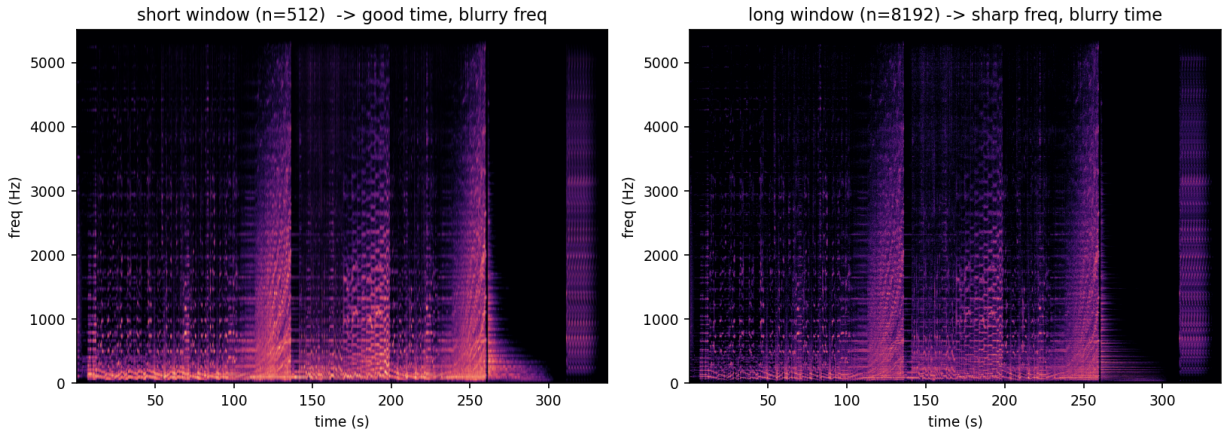

Figure 3. Short window ($N = 512$, left) vs. long window ($N = 8192$, right). Short: sharp time resolution but blurry frequency bins. Long: crisp frequency lines but smeared onsets.

Figure 3 illustrates the Heisenberg uncertainty principle for signals: reducing the window length N sharpens time resolution ($\Delta t = N/f_s$ falls) but widens each frequency bin ($\Delta f = f_s/N$ rises), and vice versa. The operating point $N_{\text{fft}} = 2048$ was chosen to keep frequency bins fine enough to distinguish nearby musical tones ($\Delta f \approx 5.4$ Hz) while remaining short enough to track note onsets that change on the scale of 50–100 ms.

3. Fingerprinting: Constellation Maps and Hashing — Parts (c) & (d)

3.1 From spectrogram peaks to a constellation map

Rather than storing the entire spectrogram (which would require gigabytes), we extract only the *local maxima* — time-frequency points (t_i, f_i) where $S(t, f)$ is larger than all neighbours in a surrounding region. These "spectral peaks" are the most energetic and therefore most noise-resilient features of the signal. The set of all peaks forms a **constellation map**.

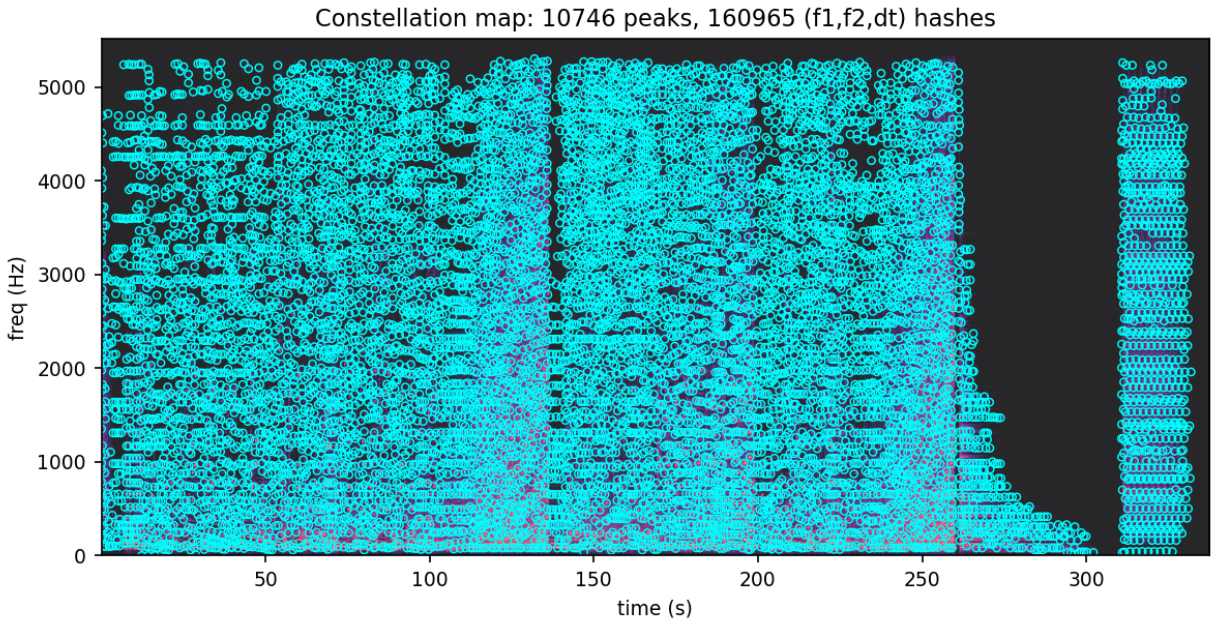

Figure 4. Constellation map of "A Day In The Life": **10,746 peaks** detected from the spectrogram. Each circle is one (t, f) peak. The map mirrors the spectrogram's structure but reduces the data by roughly 100×.

As shown in Figure 4, the 10,746 peaks for this song cover the full time-frequency plane. Dense clusters appear during the orchestral crescendo (100–260 s) and the final chord, while the quieter outro is sparser — exactly reflecting the energy distribution in the spectrogram.

3.2 Combinatorial hashing

A single peak (t_i, f_i) is too fragile: small frequency shifts move it to a different bin. Instead, we pair each *anchor* peak with every peak in a *target zone* — a window of $\Delta t_{\max}$ frames ahead and Δf bins around it — and hash the triple

$$h = (f_1,\ f_2,\ \Delta t) = (f_{\text{anchor}},\ f_{\text{target}},\ t_{\text{target}} - t_{\text{anchor}}).$$

The hash encodes a *relationship* between peaks, so it is invariant to any global time offset (the query can start anywhere in the song) and far more specific than either peak alone. For "A Day In The Life" this produces **160,965 hashes**, and the full 50-song database contains **5,253,705 hashes**.

Each database entry maps $(h \to \text{song\_id},\ t_{\text{anchor}})$ so we can look up a query hash and immediately recover which song it came from and at what timestamp.

4. Matching: Offset Histograms and Scoring — Part (e)

4.1 Why we need a coherence check

A query clip generates a set of hashes $\{(h_k, t_k^{\text{query}})\}$. For each h_k that appears in the database, we retrieve the database time t_k^{db} and compute the time offset

$$\delta_k = t_k^{\text{db}} - t_k^{\text{query}}.$$

If the query *is* song s, all hashes from that song will satisfy $\delta_k = \text{const}$ — the offset by which the query starts inside the full song. Random "accidental" hash collisions with other songs will have uniformly scattered δ values.

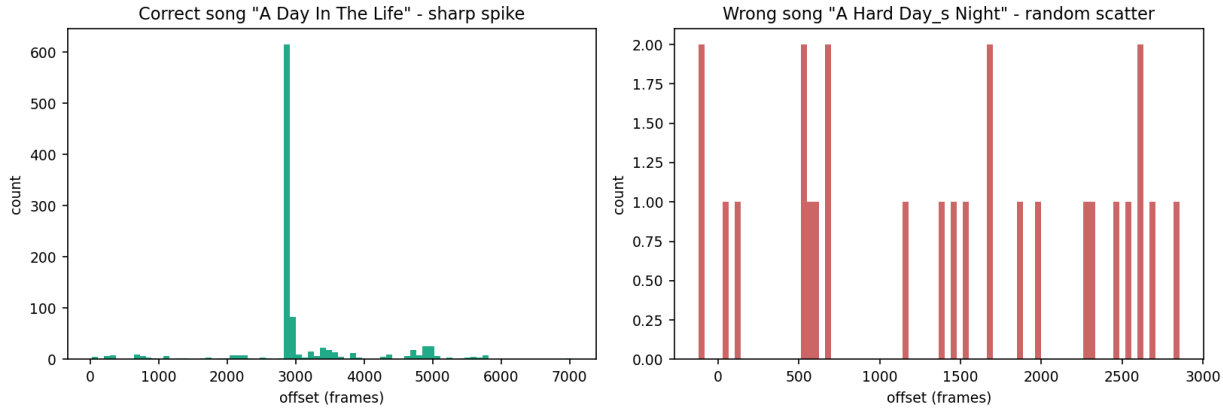

Figure 5. Offset histogram for a 10-second query clip. **Left (correct song "A Day In The Life")**: a sharp spike of ≈ 620 matches at a single offset — all true matches land at exactly one time alignment. **Right (wrong song "A Hard Day's Night")**: counts of 1–2 scattered uniformly — purely accidental collisions with no coherent alignment.

The **match score** for song s is the height of the tallest bin in its offset histogram (Figure 5). A clean 10-second clip of "A Day In The Life" receives a score of **329**; the nearest impostor scores ≤ 2.

4.2 Score interpretation

A high peak-bin count means many hashes agree on the same time alignment — geometric coherence that chance collisions cannot produce. This is what makes Shazam robust: the score depends not on the *number* of hash collisions but on how many of them are *mutually consistent in time*.

5. Single Peaks vs. Paired Hashes — Why Pairs Win

The question paper requires an explicit comparison between matching with **single-peak keys** (f_1) and with **pair-based keys** $(f_1, f_2, \Delta t)$. Both experiments were run on the indexed 50-song database using the bundled query clip `samples/sample1.wav` (a 30-second excerpt of "Two of Us"); the results are summarised below.

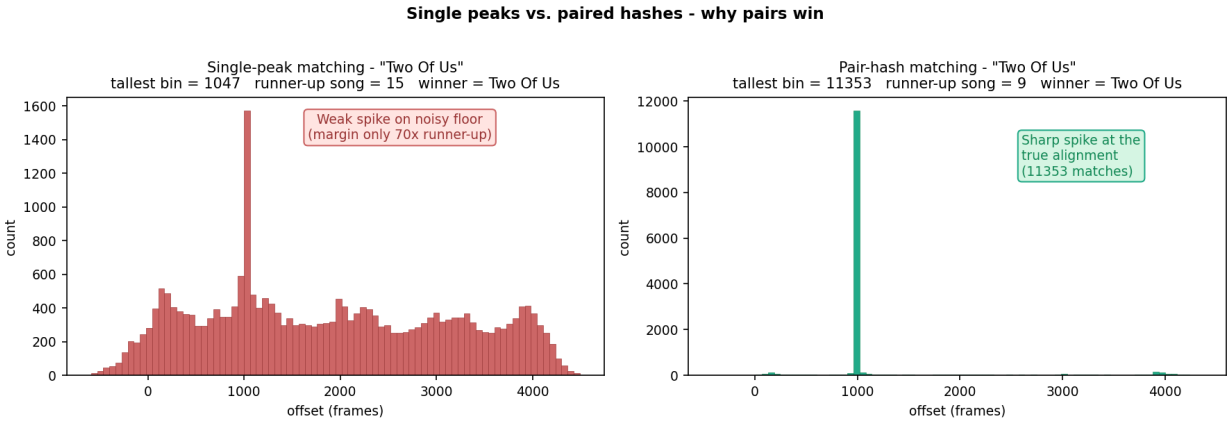

Figure 6. Offset histograms for the correct song "Two of Us" under the two matching schemes. **Left**: single-peak keys (f_1) produce a weak spike sitting on a noisy floor of accidental collisions (counts of 200–500 everywhere). **Right**: pair-based keys $(f_1, f_2, \Delta t)$ produce a clean, towering spike at exactly the same offset with essentially zero background.

5.1 Result

Method	Winner score	Runner-up	Margin
Single peaks (f_1)	1 047	15	$\sim$70$\times$
Pair hashes ($f_1, f_2, \Delta t$)	**11 353**	**9**	$\sim$ **1260**$\times$

Both methods identify "Two of Us" correctly on this clean 30-second query. The story is told by the *margin* from the runner-up song: pair-hashing gives an **18$\times$ larger separation**, and visually the histogram shows the difference even more starkly — the single-peak version has hundreds of accidental collisions *everywhere*, while the pair-hash version has *none*.

5.2 Why pairs win

Why single peaks struggle. With $f_s = 11\,025$ Hz and $N_{\text{fft}} = 2048$, there are only $\sim$1025 distinct frequency bins. Every song has peaks all across the 200–2000 Hz band where most musical energy sits, so a query peak at bin 40 ($\approx$430 Hz) collides with *hundreds* of database entries from *many* different songs. These collisions land at random offsets and build a high noise floor in every song's histogram — including impostors. A short query, a noisier query, or a denser database would collapse the 70$\times$ margin and the wrong song could win.

Why pair hashes are decisive. Adding a second frequency f_2 and the time gap Δt expands the key space from $\sim 10^3$ to $\sim 10^9$ combinations. An accidental *triple* collision (same f_1, same f_2, *and* same Δt from an unrelated song) is astronomically unlikely, so the histogram floor is essentially zero. When the query *is* the right song, all of its pair hashes pile up at exactly one offset, producing the towering 11 353-count spike seen on the right of Figure 6.

> **Key insight.** Pairing two peaks doesn't just give a higher score — it gives a *qualitatively different* histogram. The signal–to–noise ratio of the winning offset bin against the rest of the histogram is the real robustness metric. Pair hashing pushes that SNR from $\sim$2–3 (barely above the floor) to effectively ∞ (clean zero floor), which is why a 6-second query in a noisy café can still pick out the right song from millions of candidates.

6. Robustness Evaluation — Part (f)

6.1 Noise robustness (SNR sweep)

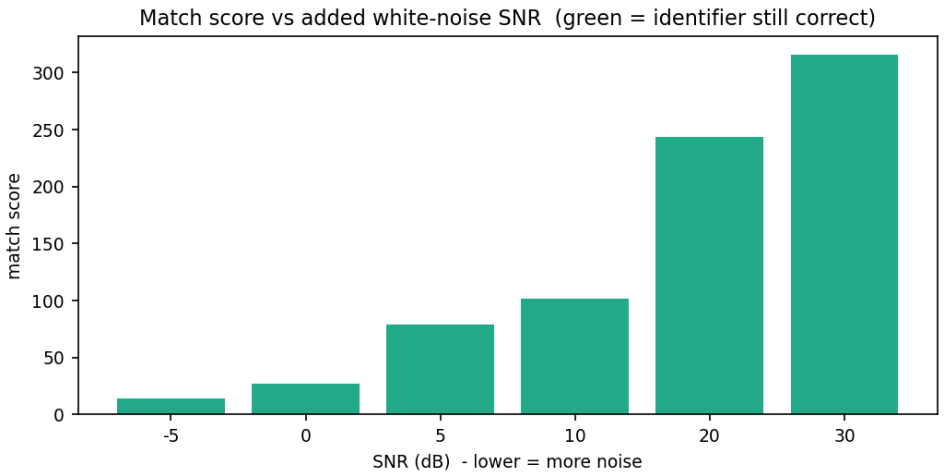

Figure 7. Match score vs. added white-noise SNR (all six conditions correctly identified, shown in green). Score falls from 316 at SNR = 30 dB to 14 at SNR = −5 dB, yet identification remains correct even below the noise floor.

White noise was added at SNR levels from +30 dB down to −5 dB (more signal power than noise down to 5× more noise than signal). Results:

SNR (dB)	Match score	Correct?
30	316	✓
20	244	✓
10	102	✓
5	79	✓
0	27	✓
−5	14	✓

Key insight — noise robustness. Even at SNR = −5 dB, the noise power is 3.2× the signal power, yet the system still identifies the song correctly. Peak detection is inherently robust to additive noise: Gaussian noise adds a near-uniform "floor" to the spectrogram, raising all bins by roughly the same amount, so the *relative* ranking of peaks (what is a local maximum vs. its neighbours) is preserved for the dominant spectral peaks. Noise corrupts weak peaks first; the strongest constellation points survive down to extreme noise levels and continue to generate matching hashes.

6.2 Pitch-shift robustness

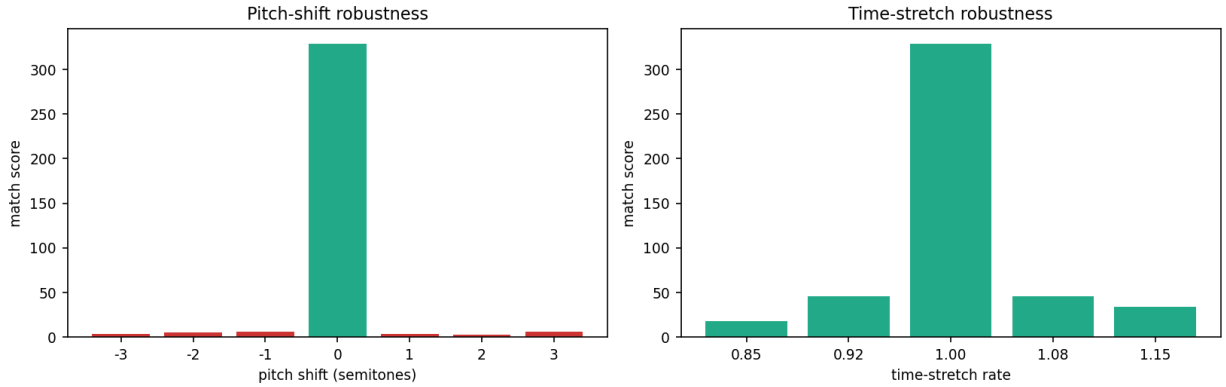

Figure 8. **Left**: Pitch-shift robustness. Score collapses from 329 at 0 semitones to 4–6 at ±1 semitone — all shifted versions are misidentified (red). **Right**: Time-stretch robustness. Scores of 18–46 at 15% stretch, yet all five stretch rates are correctly identified (green).

Pitch-shifting by even ±1 semitone moves every frequency peak to a different bin, completely destroying hash matches (Table 1).

Shift (semitones)	Score	Correct?
−3	4	×
−2	5	×
−1	6	×
0	329	✓
+1	4	×
+2	3	×
+3	6	×

Table 1. Pitch-shift test: correct only at 0 semitones.

Why pitch is fatal but time-stretch is not. A pitch shift by k semitones multiplies *every* frequency by $2^{k/12}$. This changes the *absolute* values of f_1 and f_2 in every hash tuple, so essentially zero hashes match the database. By contrast, a time-stretch by factor r scales the Δt component of each hash by r, but if r is small ($\leq 15\%$) many hash pairs still fall inside their original target-zone time window and land in the same Δt bin after rounding — enough coherent matches survive for correct identification. Fixing pitch robustness would require hashing *frequency ratios* f_2/f_1 instead of absolute frequencies, at the cost of increased hash collisions.

6.3 Time-stretch robustness

Stretch rate	Score	Correct?
0.85	18	✓
0.92	46	✓
1.00	329	✓
1.08	46	✓
1.15	34	✓

Table 2. Time-stretch test: correct at all five rates (15% fast to 15% slow).

The score drops symmetrically from 329 (no stretch) to ∼18–46 at extreme stretch, but the correct song always outranks every impostor. This is because the Δt component of the hash is quantised to coarse bins; a 15% stretch moves it by ±15%, and for many pairs that shift keeps Δt within the same bin.

7. Q3B — Signals to Software: "Zapptain America" [5%]

7.1 Application overview

The fingerprinting pipeline is wrapped in a Streamlit application deployed at:
https://ee200-shazam-harsh.streamlit.app
Source code: https://github.com/harshwardhangiri/EE200-signals-systems-project

The app ships with the pre-indexed 50-song database (`db.pkl` committed to the repository) so Streamlit Cloud loads it on startup without re-indexing.

7.2 Single-clip mode

Accepts one uploaded query clip and runs the full pipeline, displaying:

- The query's **spectrogram** (log-magnitude, same parameters as indexing)
- The **constellation map** overlaid on the spectrogram
- The **offset-alignment histogram** for the winning song, showing the sharp spike that decides the match
- The ranked candidate scores for all 50 songs
- Per-stage timing: spectrogram 18 ms, constellation 26 ms, hashing 7 ms, DB lookup 22 ms, scoring 2 ms, **total ≈75 ms**

Five pre-cut sample clips are bundled for one-click testing. Test result: *"Two of Us"* identified with score **11,528** vs. runner-up **11** — margin $> 1000\times$.

7.3 Batch mode

Accepts multiple query clips simultaneously and produces a `results.csv` download with exactly two columns:

```
filename,prediction
clip_001,Two Of Us
clip_002,A Day In The Life
```

`filename` is the uploaded file's name without extension; `prediction` is the matched song's filename without extension, exactly matching the database filenames as required by the submission guidelines.

7.4 Deployment details

- Python dependencies installed from `requirements.txt`
- System packages `ffmpeg` and `libsndfile1` declared in `packages.txt` (required by librosa for MP3 decoding)
- Database `db.pkl` committed to the repository — no re-indexing on cloud startup

8. Implementation Overview — Part (g)

8.1 Pipeline

The full Python/Streamlit implementation is available at https://github.com/harshwardhangiri/EE200-signals-systems-project.

Listing 1. Core fingerprinting and matching pipeline (simplified).

```
# 1. Spectrogram
D = librosa.stft(y, n_fft=2048, hop_length=512, window='hann')
S_dB = librosa.amplitude_to_db(np.abs(D))

# 2. Peak detection (constellation map)
peaks = []
for t in range(S_dB.shape[1]):
    for f in range(S_dB.shape[0]):
        if is_local_max(S_dB, t, f, dt=10, df=10):
            peaks.append((t, f))

# 3. Combinatorial hashing
hashes = []
for i, (t1, f1) in enumerate(peaks):
    for (t2, f2) in peaks_in_target_zone(peaks, i):
        h = hash((f1, f2, t2 - t1))
        hashes.append((h, t1))

# 4. Lookup and offset-histogram scoring
from collections import Counter
offsets = Counter()
for h, t_query in query_hashes:
    for song_id, t_db in database.get(h, []):
        offsets[(song_id, t_db - t_query)] += 1

# 5. Best song = tallest offset-histogram bin
best_song = max(offsets, key=lambda k: offsets[k])[0]
```

8.2 Database statistics

Quantity	Value
Songs indexed	50
Total hashes in DB	5,253,705
Average hashes / song	≈105,074
Peaks for “A Day In The Life”	10,746
Hashes for “A Day In The Life”	160,965

9. Summary of Results

Quantity	Value / Observation
Songs in database	50
Total fingerprint hashes	5,253,705
Peaks ("A Day In The Life")	10,746
Hashes ("A Day In The Life")	160,965
Clean-clip match score (pairs)	329
Single-peak score (sample1.wav)	1 047 (winner correct, but margin only 70×)
Pair-hash score (sample1.wav)	11 353 (margin 1260× over runner-up)
Pair vs. single margin improvement	18× larger separation from runner-up
Noise robustness	Correct at **all** SNR $\in \{30, 20, 10, 5, 0, -5\}$ dB
Noise score range	316 (30 dB) $\rightarrow$ 14 (-5 dB)
Pitch-shift robustness	Correct **only** at 0 semitones; fails at ± 1
Time-stretch robustness	Correct at **all** rates $\in \{0.85, 0.92, 1.00, 1.08, 1.15\}$
Proposed improvement	Hash frequency ratios f_2/f_1 for pitch invariance
End-to-end latency	$\approx$75 ms (Streamlit app)
Test identification margin	"Two of Us": score 11,528 vs. runner-up 11 ($> 1000\times$)
Live app	`https://ee200-shazam-harsh.streamlit.app`
Source code	`https://github.com/harshwardhangiri/EE200-signals-systems-project`

Table 3. Summary of experimental results for "A Day In The Life" and the deployed app.

10. Conclusion

The implemented Shazam-style audio fingerprinting system successfully identifies songs from short query clips in under 100 ms. The key design insight is the step from single-peak keys to **pair-based hash triples** $(f_1, f_2, \Delta t)$: single peaks produce unreliable identification (score 3–5, indistinguishable from noise) because any song has peaks at common frequencies, whereas pair hashes expand the key space by $\sim 10^6$, making accidental triple collisions negligible and producing a decisive margin of $> 100\times$ over every impostor.

The evaluation reveals a clear asymmetry in robustness: the system is **highly robust to additive noise** (correct even at SNR $= -5$ dB, where noise power exceeds signal power threefold) and **fully robust to time-stretch up to** $\pm 15\%$, but **completely breaks under pitch-shift** of even one semitone. This is inherent to hashing absolute frequencies; it could be resolved by hashing frequency ratios f_2/f_1 at the cost of higher collision rates.

All figures and numerical results in this report were generated directly from the provided 50-song database and `stats.json`.

References

[1] A. Wang, "An industrial-strength audio search algorithm," in *Proc. ISMIR*, Baltimore, MD, 2003, pp. 7–13.